



POLITÉCNICA

Universidad Politécnica de Madrid

Bases de Datos II

Práctica Cassandra: Sistema de Rankings

Diseño, implementación y consultas en Apache Cassandra

Yixuan Lu Guo
Shengkai Zhu

23 de marzo de 2026

Índice

1. Introducción	2
2. Tarea 1: Diseño de la base de datos Cassandra	2
2.1. Hall of Fame: <code>hall_of_fame_by_country</code>	2
2.2. Tabla auxiliar: <code>dungeons_by_country</code>	3
2.3. User Statistics: <code>user_statistics_by_dungeon</code>	3
2.4. Top Horde: <code>top_horde_by_event</code>	3
3. Tarea 2: Consultas SQL para exportación a CSV	4
3.1. <code>hall_of_fame_by_country.sql</code>	4
3.2. <code>dungeons_by_country.sql</code>	5
3.3. <code>user_statistics_by_dungeon.sql</code>	5
3.4. <code>top_horde_by_event.sql</code>	5
3.5. Resumen de ficheros CSV generados	5
4. Tarea 3: Clúster local de 3 nodos	6
5. Tarea 4: Creación de tablas y carga de datos (<code>leaderboards.cql</code>)	7
5.1. Keyspace	7
5.2. Creación de tablas	7
5.3. Carga de datos	8
6. Tarea 5 (Opcional): Actualización de las escrituras	8
6.1. When: User finish dungeon	9
6.2. When: User kills monster during Horde event	9
6.3. Justificación general del diseño de escrituras	10
7. Tarea 6: Consultas de lectura y escritura con niveles de consistencia	10
7.1. Niveles de consistencia elegidos	10
7.2. Lecturas en CQL	10
7.3. Escrituras en CQL	11
7.4. Implementación en Python (<code>Tarea-6.ipynb</code>)	12
7.4.1. Conexión al clúster	12
7.4.2. Lectura: Hall of Fame	12
7.4.3. Lectura: User Statistics	14
7.4.4. Lectura: Top Horde	14
7.4.5. Escritura: User finish dungeon	15
7.4.6. Escritura: User kills monster during Horde	17
8. Conclusiones	18

1. Introducción

En el videojuego *Jotun's Lair* de la empresa *Norsewind Studios*, existen tres leaderboards que requieren una base de datos eficiente y escalable:

- **Hall of Fame:** para un país concreto, muestra para cada mazmorra del juego el TOP 5 de jugadores más rápidos de ese país, incluyendo sus tiempos.
- **User Statistics:** muestra los tiempos que un jugador ha tardado en completar una mazmorra en particular, ordenados de menor a mayor.
- **Top Horde:** muestra los N jugadores que más monstruos han matado durante un evento de Horda concreto, local por país. Este leaderboard se actualiza en tiempo real durante el gameplay.

La base de datos relacional existente no cumple con los requisitos de rendimiento y escalabilidad, en especial para los nuevos eventos de Horda. Por ello, se plantea migrar a Apache Cassandra. Esta memoria documenta el diseño, la implementación y la justificación de todas las decisiones tomadas a lo largo de las distintas tareas.

2. Tarea 1: Diseño de la base de datos Cassandra

El diseño sigue el principio fundamental de Cassandra: **modelar las tablas según las queries**, no según las entidades del modelo relacional. Cada lectura del juego se traduce en una tabla (o conjunto de tablas) optimizada para responderla accediendo a una sola partición, sin necesidad de JOINS.

2.1. Hall of Fame: hall_of_fame_by_country

Lectura: dado un país, mostrar para cada mazmorra el TOP 5 de jugadores más rápidos.

Decisiones de diseño:

La clave de partición es (**country**, **dungeon_id**). Al incluir ambos campos, cada partición contiene exactamente los datos de una mazmorra en un país concreto, de modo que la consulta se resuelve en un único nodo. Esto nos permite aplicar LIMIT 5 directamente en CQL para obtener el TOP 5 sin necesidad de procesamiento adicional en la aplicación.

Las claves de clustering son (**time_minutes** ASC, **email** ASC). El tiempo se ordena de forma ascendente para que los jugadores más rápidos aparezcan primero. El email se añade como segunda clustering column para garantizar la unicidad de cada fila: si dos jugadores completan la mazmorra en el mismo tiempo, sus registros no se sobrescriben entre sí.

Se declara **dungeon_name** como STATIC porque dentro de una partición (**country**, **dungeon_id**) el nombre de la mazmorra es siempre el mismo. De esta forma, el nombre solo se almacena una vez por partición en lugar de repetirse en cada fila del TOP 5.

Listing 1: Tabla hall_of_fame_by_country

```
CREATE TABLE hall_of_fame_by_country (  
  country TEXT,  
  dungeon_id INT,  
  time_minutes INT,  
  email TEXT,  
  user_name TEXT,  
  date TIMESTAMP,  
  dungeon_name TEXT STATIC,  
  PRIMARY KEY ((country, dungeon_id), time_minutes, email)  
) WITH CLUSTERING ORDER BY (time_minutes ASC, email ASC);
```

2.2. Tabla auxiliar: dungeons_by_country

Como la partition key de `hall_of_fame_by_country` incluye tanto `country` como `dungeon_id`, Cassandra no permite hacer `WHERE country = ?` sin especificar también el `dungeon_id`. Sin embargo, el enunciado pide que al introducir un país se muestren *todas* sus mazmorras.

Para resolver esto, creamos una tabla auxiliar que, dado un país, devuelve todos los `dungeon_id` disponibles. La aplicación primero consulta esta tabla y después itera sobre cada mazmorra para obtener su TOP 5 correspondiente.

Listing 2: Tabla `dungeons_by_country`

```
CREATE TABLE dungeons_by_country (
  country TEXT,
  dungeon_id INT,
  PRIMARY KEY ((country), dungeon_id)
) WITH CLUSTERING ORDER BY (dungeon_id ASC);
```

2.3. User Statistics: user_statistics_by_dungeon

Lectura: dado un email y un `dungeon_id`, mostrar todos los tiempos del jugador ordenados de menor a mayor.

La clave de partición es `(email, dungeon_id)`, que coincide exactamente con los parámetros de entrada de la lectura. Así, todos los intentos de un jugador en una mazmorra concreta residen en la misma partición. Las claves de clustering son `(time_minutes ASC, date ASC)`: el tiempo ordena los intentos del más rápido al más lento, y la fecha permite distinguir intentos con el mismo tiempo (por ejemplo, si un jugador completa la misma mazmorra dos veces en 20 minutos pero en fechas distintas).

Listing 3: Tabla `user_statistics_by_dungeon`

```
CREATE TABLE user_statistics_by_dungeon (
  email TEXT,
  dungeon_id INT,
  time_minutes INT,
  date TIMESTAMP,
  PRIMARY KEY ((email, dungeon_id), time_minutes, date)
) WITH CLUSTERING ORDER BY (time_minutes ASC, date ASC);
```

2.4. Top Horde: top_horde_by_event

Lectura: dado un país, un `event_id` y un K, mostrar los K jugadores con más kills.

La clave de partición es `(country, event_id)`, ya que por temas de latencia las Hordas son locales por país. Las claves de clustering son `(n_killed DESC, email ASC)`: los jugadores con más kills aparecen primero gracias al orden descendente, y el email garantiza la unicidad de cada fila.

Problema de actualización: como `n_killed` forma parte de la clave primaria, no se puede modificar su valor directamente con un `UPDATE`. En Cassandra, cambiar un componente de la clave equivale a insertar una nueva fila (y la antigua seguiría existiendo si no la eliminamos). La estrategia que adoptamos es hacer un `DELETE` de la fila con el `n_killed` antiguo seguido de un `INSERT` con el nuevo valor, agrupados en un `BATCH` para reducir la ventana de inconsistencia. Durante ese breve intervalo el ranking puede “bailar” ligeramente, lo cual el propio enunciado considera aceptable e incluso positivo para la motivación de los jugadores.

Para poder obtener el `n_killed` actual de un jugador sin tener que escanear toda la partición (ya que `email` no es la primera clustering column), se crea un **índice secundario** sobre la columna `email`. Aunque los índices secundarios en Cassandra generan queries distribuidas entre

nodos, en el contexto de la Horda — donde la consistencia no es crítica y se usa `CONSISTENCY ONE`— es un compromiso aceptable para simplificar el modelo de datos.

Nota: somos conscientes de que en sistemas a gran escala (estilo Netflix o Discord) se preferiría crear dos tablas separadas: una tabla rápida para buscar al usuario por email y otra tabla ordenada para el leaderboard. Sin embargo, para este prototipo el índice secundario cumple perfectamente el objetivo sin añadir complejidad innecesaria al modelo.

Listing 4: Tabla `top_horde_by_event` con índice secundario

```
CREATE TABLE top_horde_by_event (
  country TEXT,
  event_id INT,
  n_killed INT,
  user_name TEXT,
  email TEXT,
  PRIMARY KEY ((country, event_id), n_killed, email)
) WITH CLUSTERING ORDER BY (n_killed DESC, email ASC);

-- Índice secundario para buscar por email sin
-- especificar n_killed
CREATE INDEX IF NOT EXISTS idx_email_horde
ON top_horde_by_event (email);
```

¿Por qué no usamos tablas `COUNTER`? Se podría pensar en utilizar columnas de tipo `COUNTER` para contar las kills de cada jugador. Sin embargo, las tablas `COUNTER` en Cassandra tienen restricciones importantes: no pueden mezclar columnas `COUNTER` con columnas normales (salvo las que forman la clave primaria), y solo soportan `UPDATE` con incrementos, no `INSERT` ni `DELETE`. Esto haría imposible mantener `user_name` en la misma tabla e imposibilitaría el patrón `DELETE + INSERT` que necesitamos. Por estas razones descartamos este enfoque.

3. Tarea 2: Consultas SQL para exportación a CSV

Para migrar los datos de la base de datos relacional a Cassandra, se crean consultas SQL que exportan los datos al formato CSV que espera cada tabla. Las fechas se formatean en ISO 8601 con `DATE_FORMAT` para que sean compatibles con el tipo `TIMESTAMP` de Cassandra.

3.1. `hall_of_fame_by_country.sql`

Esta consulta combina las tablas `completeddungeons`, `webuser` y `dungeon` para obtener, por cada intento completado, los datos necesarios para la tabla de Cassandra. Se usa `DISTINCT` para evitar duplicados en caso de que existan en la BD relacional.

Listing 5: Query de exportación para `hall_of_fame_by_country`

```
SELECT DISTINCT
  country,
  completeddungeons.idD AS dungeon_id,
  time AS time_minutes,
  webuser.email,
  userName AS user_name,
  DATE_FORMAT(date, '%Y-%m-%dT%H:%i:%s') AS date,
  dungeon.name AS dungeon_name
FROM completeddungeons
JOIN webuser ON completeddungeons.email = webuser.email
JOIN dungeon ON dungeon.idD = completeddungeons.idD;
```

3.2. dungeons_by_country.sql

Simplemente extrae las combinaciones únicas de país y mazmorra a partir de los intentos completados. Así sabemos qué mazmorras tienen datos para cada país.

Listing 6: Query de exportación para dungeons_by_country

```
SELECT DISTINCT
    country, idD AS dungeon_id
FROM completeddungeons
JOIN webuser ON webuser.email = completeddungeons.email;
```

3.3. user_statistics_by_dungeon.sql

Aquí no hace falta JOIN con webuser porque la tabla de estadísticas solo necesita el email (que ya está en completeddungeons), el id de la mazmorra, el tiempo y la fecha.

Listing 7: Query de exportación para user_statistics_by_dungeon

```
SELECT
    email,
    idD AS dungeon_id,
    time AS time_minutes,
    DATE_FORMAT(date, '%Y-%m-%dT%H:%i:%s') AS date
FROM completeddungeons;
```

3.4. top_horde_by_event.sql

Para las Hordas, la consulta agrupa las kills por evento y usuario, contando cuántos monstruos ha matado cada jugador en cada evento. Se incluye el país y el nombre de usuario de forma denormalizada.

Listing 8: Query de exportación para top_horde_by_event

```
SELECT
    webuser.country,
    event.idE AS event_id,
    COUNT(kills.idK) AS n_killed,
    webuser.userName AS user_name,
    webuser.email
FROM event
JOIN kills ON event.idE = kills.idE
JOIN webuser ON kills.email = webuser.email
GROUP BY event.idE, webuser.email,
    webuser.country, webuser.userName;
```

3.5. Resumen de ficheros CSV generados

Los CSV resultantes tienen las siguientes cabeceras, que coinciden exactamente con las columnas de las tablas Cassandra:

Fichero CSV	Cabeceras
hall_of_fame_by_country.csv	country, dungeon_id, time_minutes, email, user_name, date, dungeon
dungeons_by_country.csv	country, dungeon_id
user_statistics_by_dungeon.csv	email, dungeon_id, time_minutes, date
top_horde_by_event.csv	country, event_id, n_killed, user_name, email

Cuadro 1: Cabeceras de los ficheros CSV exportados

4. Tarea 3: Clúster local de 3 nodos

Se prepara un clúster local de Cassandra con Docker Compose. Los tres nodos (`cassandra-n1`, `cassandra-n2`, `cassandra-n3`) están configurados en el mismo datacenter (`dc1`) y rack (`rack1`), todos dentro del clúster `cassandra-lab`, usando el nodo 1 como seed para que los demás se unan automáticamente.

El arranque es secuencial: cada nodo espera a que el anterior esté sano (mediante healthcheck con `cqlsh`) antes de iniciar, lo que evita problemas de formación del clúster. Además, se incluye un contenedor auxiliar `cassandra-init` que ejecuta automáticamente el fichero `leaderboards.cql` una vez que los tres nodos están operativos.

Listing 9: Extracto del `docker-compose.yml` (nodo 1)

```
services:
  cassandra-n1:
    image: cassandra:latest
    container_name: cassandra-n1
    environment:
      - CASSANDRA_CLUSTER_NAME=cassandra-lab
      - CASSANDRA_SEEDS=cassandra-n1
      - CASSANDRA_DC=dc1
      - CASSANDRA_RACK=rack1
      - CASSANDRA_ENDPOINT_SNITCH=GossipingPropertyFileSnitch
      - MAX_HEAP_SIZE=512M
      - HEAP_NEWSIZE=128M
    ports:
      - "9042:9042"
    healthcheck:
      test: ["CMD-SHELL",
            "cqlsh -e 'describe cluster' || exit 1"]
      interval: 30s
      timeout: 10s
      retries: 10
      start_period: 60s
```

Los nodos 2 y 3 tienen la misma configuración pero con `depends_on` apuntando al nodo anterior (`condition: service_healthy`). El contenedor de inicialización monta los volúmenes del CQL y los CSV:

```
cassandra-init:
  image: cassandra:latest
  depends_on:
    cassandra-n1: { condition: service_healthy }
    cassandra-n2: { condition: service_healthy }
    cassandra-n3: { condition: service_healthy }
  volumes:
    - ../leaderboards.cql:/workspace/leaderboards.cql
    - ../csv_tablas:/workspace/csv_tablas
  working_dir: /workspace
  command: >
    /bin/bash -c "
    echo 'All nodes healthy, running init...';
    cqlsh cassandra-n1 -f leaderboards.cql &&
    echo 'Init completed successfully'"
```

Para arrancar y verificar el clúster:

```
docker compose up -d
docker exec -it cassandra-n1 nodetool status
```

El comando `nodetool status` debe mostrar 3 nodos con estado UN (Up/Normal) en `dc1/rack1`.

5. Tarea 4: Creación de tablas y carga de datos (`leaderboards.cql`)

El fichero `leaderboards.cql` contiene toda la inicialización de la base de datos: creación del keyspace, creación de las cuatro tablas, el índice secundario y la carga de los CSV.

5.1. Keyspace

Listing 10: Creación del keyspace

```
CREATE KEYSPACE leaderboards
WITH replication = {
    'class': 'SimpleStrategy',
    'replication_factor': 2
};

USE leaderboards;
```

Se usa `SimpleStrategy` porque todos los nodos están en el mismo datacenter. El factor de replicación es 2, lo que significa que cada partición se replica en 2 de los 3 nodos. Si fuese un despliegue multi-datacenter, habría que cambiar a `NetworkTopologyStrategy`.

5.2. Creación de tablas

Se crean las cuatro tablas diseñadas en la Tarea 1, junto con el índice secundario sobre `email` en `top_horde_by_event`:

Listing 11: Fichero `leaderboards.cql` completo

```
CREATE TABLE hall_of_fame_by_country (
    country TEXT,
    dungeon_id INT,
    time_minutes INT,
    email TEXT,
    user_name TEXT,
    date TIMESTAMP,
    dungeon_name TEXT STATIC,
    PRIMARY KEY ((country, dungeon_id),
                 time_minutes, email)
) WITH CLUSTERING ORDER BY
    (time_minutes ASC, email ASC);

CREATE TABLE dungeons_by_country (
    country TEXT,
    dungeon_id INT,
    PRIMARY KEY ((country), dungeon_id)
) WITH CLUSTERING ORDER BY (dungeon_id ASC);

CREATE TABLE user_statistics_by_dungeon (
    email TEXT,
    dungeon_id INT,
    time_minutes INT,
    date TIMESTAMP,
    PRIMARY KEY ((email, dungeon_id),
                 time_minutes, date)
) WITH CLUSTERING ORDER BY
```



```

        (time_minutes ASC, date ASC);

CREATE TABLE top_horde_by_event (
    country TEXT,
    event_id INT,
    n_killed INT,
    user_name TEXT,
    email TEXT,
    PRIMARY KEY ((country, event_id),
                 n_killed, email)
) WITH CLUSTERING ORDER BY
    (n_killed DESC, email ASC);

CREATE INDEX IF NOT EXISTS idx_email_horde
ON top_horde_by_event (email);

```

5.3. Carga de datos

Tras crear cada tabla, se cargan los datos desde los CSV generados en la Tarea 2 mediante COPY FROM:

Listing 12: Carga de datos con COPY FROM

```

COPY hall_of_fame_by_country
    (country, dungeon_id, time_minutes, email,
     user_name, date, dungeon_name)
FROM 'csv_tablas/hall_of_fame_by_country.csv'
WITH DELIMITER = ',' AND HEADER = TRUE;

COPY dungeons_by_country (country, dungeon_id)
FROM 'csv_tablas/dungeons_by_country.csv'
WITH DELIMITER = ',' AND HEADER = TRUE;

COPY user_statistics_by_dungeon
    (email, dungeon_id, time_minutes, date)
FROM 'csv_tablas/user_statistics_by_dungeon.csv'
WITH DELIMITER = ',' AND HEADER = TRUE;

COPY top_horde_by_event
    (country, event_id, n_killed, user_name, email)
FROM 'csv_tablas/top_horde_by_event.csv'
WITH DELIMITER = ',' AND HEADER = TRUE;

```

Las rutas de los CSV (csv_tablas/...) coinciden con el volumen montado en el contenedor cassandra-init del docker-compose (../csv_tablas:/workspace/csv_tablas).

6. Tarea 5 (Opcional): Actualización de las escrituras

Las escrituras originales del enunciado no contienen toda la información necesaria para escribir directamente en nuestras tablas Cassandra sin hacer lecturas adicionales. En Cassandra no hay JOINS, así que cada evento de escritura debe traer toda la información denormalizada que la tabla necesita.

6.1. When: User finish dungeon

Campo	Original	Actualizado
dungeon_id		
email		
time_minutes		
date		
country	—	(nuevo)
dungeon_name	—	(nuevo)
user_name	—	(nuevo)

Cuadro 2: Campos de escritura para “User finish dungeon”

Justificación de cada campo nuevo:

- **country**: es parte de la partition key de `hall_of_fame_by_country` (partición (`country`, `dungeon_id`)) y de `dungeons_by_country` (partición (`country`)). Sin este campo, no se puede escribir en la partición correcta sin consultar previamente la tabla de usuarios.
- **dungeon_name**: se devuelve en la lectura del Hall of Fame y se almacena como `STATIC` en `hall_of_fame_by_country`. Sin él, el backend tendría que consultar la tabla relacional `Dungeon`.
- **user_name**: aparece en el output del Hall of Fame (Top 5). Se guarda denormalizado para evitar lecturas a `WebUser`.

Este evento dispara escrituras en **tres tablas**: `hall_of_fame_by_country`, `user_statistics_by_dungeon` y `dungeons_by_country` (esta última de forma idempotente: si la combinación país-mazmorra ya existe, Cassandra simplemente sobrescribe con los mismos valores).

6.2. When: User kills monster during Horde event

Campo	Original	Actualizado
event_id		
email		
monster_id		(opcional, útil para logging)
country	—	(nuevo)
user_name	—	(nuevo)

Cuadro 3: Campos de escritura para “User kills monster”

Justificación: `country` es necesario porque `top_horde_by_event` particiona por (`country`, `event_id`) y el ranking de Horda es local por país. `user_name` se devuelve en la lectura del Top Horde y se guarda denormalizado. No incluimos `monster_id` como campo obligatorio ya que no es necesario para el ranking (solo serviría para logging), y así evitamos operaciones innecesarias.

En cuanto al `n_killed`, no es un campo que envíe el cliente del juego: se calcula en el backend consultando el valor actual mediante el índice secundario sobre `email` y sumándole 1.

Flujo de actualización:

1. Leer el `n_killed` actual del jugador usando el índice secundario sobre `email`.
2. Calcular `new_n_killed = old_n_killed + 1` en el backend.
3. En un `BATCH`: borrar la fila con el `n_killed` antiguo e insertar la nueva fila con el `new_n_killed`.

6.3. Justificación general del diseño de escrituras

1. Los leaderboards son locales por país, y nuestro diseño usa `country` como parte de la clave de partición en `hall_of_fame_by_country` y `top_horde_by_event`. Sin `country` en la escritura, no se puede escribir en la partición correcta sin consultar la tabla de usuarios.
2. Queremos que cada evento de escritura se traduzca en escrituras directas sobre las tablas denormalizadas. Si faltan `country`, `user_name` o `dungeon_name`, el backend tendría que hacer lecturas previas.
3. `dungeon_name` se almacena como `STATIC` en `hall_of_fame_by_country` para no repetir el mismo nombre en cada fila del Top 5; se guarda una sola vez por partición.
4. Para las Hordas, el enunciado dice que la latencia durante gameplay es crítica. Evitar lecturas adicionales por cada kill es esencial; con `country` y `user_name` en el evento, la actualización es inmediata.

7. Tarea 6: Consultas de lectura y escritura con niveles de consistencia

7.1. Niveles de consistencia elegidos

Leaderboard	Consistencia	Justificación
Hall of Fame	QUORUM	La consistencia es importante y un pequeño retardo no afecta, ya que los jugadores están navegando por un menú en el campamento, no en pleno gameplay.
User Statistics	QUORUM	Misma situación: se consulta desde el campamento. La consistencia importa y la latencia no es crítica.
Top Horde	ONE	La latencia es crítica porque se ejecuta durante gameplay en tiempo real. La consistencia es menos importante: el enunciado indica que si los rankings “bailan un poco”, los jugadores se motiven más.

Cuadro 4: Niveles de consistencia por leaderboard

Nota sobre QUORUM con RF=2: con un factor de replicación de 2 y `CONSISTENCY QUORUM`, se necesita respuesta de $\lceil 2/2 \rceil = 1$ réplica, lo cual en la práctica es equivalente a `ONE`. Sin embargo, optamos por declarar `QUORUM` igualmente porque escala mejor: si en producción se aumenta el factor de replicación a 3, `QUORUM` pasaría a requerir 2 de 3 réplicas, manteniendo la consistencia fuerte automáticamente sin necesidad de modificar el código.

7.2. Lecturas en CQL

Listing 13: Lectura del Hall of Fame

```
CONSISTENCY QUORUM;  
  
-- Paso 1: obtener las mazmorras del país  
SELECT dungeon_id FROM dungeons_by_country  
WHERE country = 'es_ES';
```

```
-- Paso 2: para cada dungeon_id, obtener el TOP 5
SELECT dungeon_id, dungeon_name, time_minutes,
       user_name, email, date
FROM hall_of_fame_by_country
WHERE country = 'es_ES' AND dungeon_id = 1
LIMIT 5;
```

Listing 14: Lectura de User Statistics

```
CONSISTENCY QUORUM;

SELECT time_minutes, date
FROM user_statistics_by_dungeon
WHERE email = 'aabe@example.net' AND dungeon_id = 0;
```

Listing 15: Lectura del Top Horde

```
CONSISTENCY ONE;

SELECT n_killed, user_name, email
FROM top_horde_by_event
WHERE country = 'de_DE' AND event_id = 0
LIMIT 5;
```

7.3. Escrituras en CQL

Listing 16: Escritura: User finish dungeon

```
CONSISTENCY QUORUM;

BEGIN BATCH
  -- Registrar combinacion pais-mazmorra (idempotente)
  INSERT INTO dungeons_by_country (country, dungeon_id)
  VALUES ('ES', 101);

  -- Insertar en el Hall of Fame
  INSERT INTO hall_of_fame_by_country
  (country, dungeon_id, time_minutes, email,
   user_name, date, dungeon_name)
  VALUES ('ES', 101, 11, 'player@email.com',
          'Thor', '2026-03-16T14:06:11Z',
          'Cueva Helada');

  -- Insertar en las estadisticas del usuario
  INSERT INTO user_statistics_by_dungeon
  (email, dungeon_id, time_minutes, date)
  VALUES ('player@email.com', 101, 11,
          '2026-03-16T14:06:11Z');
APPLY BATCH;
```

Usamos un LOGGED BATCH (el tipo por defecto) porque las tres tablas afectadas corresponden al mismo evento lógico y queremos que se ejecuten de forma atómica. El campo `date` es `TIMESTAMP` en nuestras tablas, por lo que el backend convierte la fecha ISO 8601 a un formato compatible.

Listing 17: Escritura: User kills monster during Horde

```
CONSISTENCY ONE;
```

```

-- Paso 1: Leer n_killed actual (indice secundario)
SELECT n_killed FROM top_horde_by_event
WHERE country = 'de_DE' AND event_id = 0
      AND email = 'kabuskarin@example.net';

-- Paso 2: backend calcula new_n_killed = 11 + 1 = 12
-- Paso 3: DELETE + INSERT en BATCH
BEGIN UNLOGGED BATCH
  DELETE FROM top_horde_by_event
  WHERE country = 'de_DE' AND event_id = 0
        AND n_killed = 11
        AND email = 'kabuskarin@example.net';

  INSERT INTO top_horde_by_event
  (country, event_id, n_killed, user_name, email)
  VALUES ('de_DE', 0, 12, 'trommleremilie',
          'kabuskarin@example.net');
APPLY BATCH;

```

Para la Horda usamos un UNLOGGED BATCH: ambas operaciones (DELETE + INSERT) afectan a la misma partición (country, event_id), por lo que el batch se ejecuta de forma atómica sin necesidad del log distribuido. Esto reduce la sobrecarga de escritura, algo especialmente relevante dado que la latencia durante gameplay es crítica.

7.4. Implementación en Python (Tarea-6.ipynb)

Se implementan todas las lecturas y escrituras en Python usando el driver `cassandra-driver`. La consistencia se configura individualmente en cada `SimpleStatement` o `BatchStatement`. A continuación se muestra el código completo con las salidas obtenidas al ejecutarlo contra el clúster.

7.4.1. Conexión al clúster

Listing 18: Conexión a Cassandra

```

from cassandra.cluster import Cluster
from cassandra.query import SimpleStatement, BatchStatement
from cassandra import ConsistencyLevel
import datetime
import pandas as pd

cluster = Cluster(['localhost'], port=9042)
session = cluster.connect('leaderboards')

```

7.4.2. Lectura: Hall of Fame

El Hall of Fame requiere dos pasos: primero se consultan las mazmorras del país y después se obtiene el TOP 5 de cada una. Se implementan tres funciones auxiliares:

Listing 19: Funciones para el Hall of Fame

```

def id_dungeons_of_country(session, country):
    query = "SELECT dungeon_id FROM dungeons_by_country WHERE country = %s;"
    statement = SimpleStatement(
        query, consistency_level=ConsistencyLevel.QUORUM)
    resultados = session.execute(statement, [country])
    dungeons_id = []

```

```

for fila in resultados:
    dungeons_id.append(fila.dungeon_id)
return dungeons_id

def top_by_dungeon_and_country(session, country,
                               dungeon_id, k=5):
    query = """SELECT dungeon_id, dungeon_name,
                    time_minutes, user_name, email, date
                    FROM hall_of_fame_by_country
                    WHERE country = %s AND dungeon_id = %s
                    LIMIT %s;"""
    statement = SimpleStatement(
        query, consistency_level=ConsistencyLevel.QUORUM)
    resultados = session.execute(
        statement, [country, dungeon_id, k])

    dungeon_id = resultados[0].dungeon_id
    dungeon_name = resultados[0].dungeon_name
    top = []
    for fila in resultados:
        top.append({
            'email': fila.email,
            'user_name': fila.user_name,
            'time_minutes': fila.time_minutes,
            'date': fila.date.isoformat()
        })
    return {'dungeon_id': dungeon_id,
            'dungeon_name': dungeon_name,
            f'top_{k}': top}

def hall_of_fame(session, country):
    dungeon_ids = id_dungeons_of_country(
        session, country)
    tops_pais = []
    for dungeon_id in dungeon_ids:
        tops_pais.append(
            top_by_dungeon_and_country(
                session, country, dungeon_id, 5))
    return tops_pais

```

Ejemplo de ejecución: al consultar las mazmorras disponibles para España obtenemos 20 mazmorras (ids del 0 al 19), y el TOP 5 de la mazmorra 1 devuelve:

Listing 20: Salida de `top_by_dungeon_and_country('es_ES', 1, 5)`

```

{'dungeon_id': 1,
 'dungeon_name': 'Burgstream, Culverts of the Bashful
                  Sumo Wrestlers',
 'top_5': [
     {'email': 'abellanjulio@example.org',
      'user_name': 'garciamartirio',
      'time_minutes': 0,
      'date': '2018-12-04T02:45:50'},
     {'email': 'agulloricarda@example.org',
      'user_name': 'angelino53',
      'time_minutes': 0,
      'date': '2012-10-08T01:21:09'},
     {'email': 'amadorbarbero@example.com',
      'user_name': 'sbarba',
      'time_minutes': 0,
      'date': '2022-10-10T03:12:30'},

```

```
{'email': 'berta74@example.com',
  'user_name': 'aparicioromulo',
  'time_minutes': 0,
  'date': '2016-09-26T04:29:39'},
{'email': 'brionesjose-antonio@example.com',
  'user_name': 'brumaricruz',
  'time_minutes': 0,
  'date': '2022-03-11T00:22:35'}}
```

Los datos vienen ya ordenados por `time_minutes` ASC gracias a la clustering column, sin necesidad de ordenar en la aplicación.

7.4.3. Lectura: User Statistics

Listing 21: Función para User Statistics

```
def user_statistics(session, email, dungeon_id):
    query = """SELECT time_minutes, date
                FROM user_statistics_by_dungeon
                WHERE email = %s
                AND dungeon_id = %s;"""
    statement = SimpleStatement(
        query, consistency_level=ConsistencyLevel.QUORUM)
    resultados = session.execute(
        statement, [email, dungeon_id])
    stats = []
    for fila in resultados:
        stats.append({
            'time_minutes': fila.time_minutes,
            'date': fila.date.isoformat()
        })
    return stats
```

Ejemplo de ejecución: al consultar las estadísticas del usuario `aabe@example.net` para la mazmorra 0, obtenemos sus 12 intentos ordenados de menor a mayor tiempo:

Listing 22: Salida de `user_statistics('aabe@example.net', 0)`

```
[{'time_minutes': 6, 'date': '2022-04-11T10:43:14'},
 {'time_minutes': 19, 'date': '2021-01-18T18:50:53'},
 {'time_minutes': 20, 'date': '2020-04-18T04:46:20'},
 {'time_minutes': 20, 'date': '2021-06-21T05:53:27'},
 {'time_minutes': 25, 'date': '2021-11-03T11:29:36'},
 {'time_minutes': 26, 'date': '2022-07-17T15:34:08'},
 {'time_minutes': 27, 'date': '2022-06-05T04:55:02'},
 {'time_minutes': 35, 'date': '2022-07-13T18:17:30'},
 {'time_minutes': 38, 'date': '2020-09-06T11:36:08'},
 {'time_minutes': 40, 'date': '2020-12-05T08:54:05'},
 {'time_minutes': 40, 'date': '2022-05-09T12:28:38'},
 {'time_minutes': 43, 'date': '2022-09-12T23:54:05'}]
```

Se puede apreciar cómo los tiempos salen ordenados ascendentemente y cómo los empates (por ejemplo, los dos intentos de 20 minutos) se desempatan por fecha.

7.4.4. Lectura: Top Horde

Listing 23: Función para Top Horde

```
def top_horde(session, country, event_id, K):
    query = """SELECT email, user_name, n_killed
```

```

        FROM top_horde_by_event
        WHERE country = %s AND event_id = %s
        LIMIT %s;"""
statement = SimpleStatement(
    query, consistency_level=ConsistencyLevel.ONE)
resultados = session.execute(
    statement, [country, event_id, K])
top = []
for fila in resultados:
    top.append({
        'email': fila.email,
        'user_name': fila.user_name,
        'n_killed': fila.n_killed
    })
return top

```

Ejemplo de ejecución: el TOP 5 del evento 0 en Alemania devuelve los jugadores con más kills, ya ordenados por n_killed DESC:

Listing 24: Salida de top_horde('de_DE', 0, 5)

```

[{'email': 'xullrich@example.org',
  'user_name': 'hartmann gloria',
  'n_killed': 19},
 {'email': 'silvesterconradi@example.net',
  'user_name': 'bernhard mangold',
  'n_killed': 15},
 {'email': 'gretelholsten@example.com',
  'user_name': 'johannekruschwitz',
  'n_killed': 14},
 {'email': 'pero13@example.net',
  'user_name': 'zdravko34',
  'n_killed': 14},
 {'email': 'roehrichtpaulina@example.org',
  'user_name': 'oderwaldgabriella',
  'n_killed': 14}]

```

7.4.5. Escritura: User finish dungeon

Listing 25: Función user_finish_dungeon

```

def user_finish_dungeon(session, country, dungeon_id,
                        time_minutes, email, user_name,
                        date, dungeon_name):
    batch = BatchStatement(
        consistency_level=ConsistencyLevel.QUORUM)

    # Registrar la combinacion pais-mazmorra (idempotente)
    insert_dungeons = """
        INSERT INTO dungeons_by_country
        (country, dungeon_id) VALUES (%s, %s)
    """

    # Insertar en el Hall of Fame
    # dungeon_name es STATIC: solo se almacena una vez
    # por particion (country, dungeon_id)
    insert_hall = """
        INSERT INTO hall_of_fame_by_country
        (country, dungeon_id, time_minutes, email,
         user_name, date, dungeon_name)
    """

```



```

VALUES (%s, %s, %s, %s, %s, %s, %s)
"""

# Insertar en las estadísticas del usuario
insert_stats = """
INSERT INTO user_statistics_by_dungeon
(email, dungeon_id, time_minutes, date)
VALUES (%s, %s, %s, %s)
"""

batch.add(insert_dungeons, (country, dungeon_id))
batch.add(insert_hall, (country, dungeon_id,
                        time_minutes, email, user_name, date,
                        dungeon_name))
batch.add(insert_stats, (email, dungeon_id,
                        time_minutes, date))

session.execute(batch)
print(f"Record de {user_name} guardado con exito "
      f"en las tres tablas.")

```

Ejemplo de ejecución:

```

user_finish_dungeon(
    session=session, country='ES', dungeon_id=101,
    time_minutes=11, email='player@email.com',
    user_name='Thor',
    date=datetime.datetime.now().isoformat(),
    dungeon_name='Cueva Helada')

```

Record de Thor guardado con exito en las tres tablas.

Si después verificamos que la escritura se ha propagado correctamente consultando las estadísticas del usuario recién insertado y el Hall of Fame del país 'ES':

Listing 26: Verificación: estadísticas de player@email.com en mazmorra 101

```

[{'time_minutes': 11, 'date': '2026-03-16T14:06:11.014000'},
 {'time_minutes': 11, 'date': '2026-03-16T16:30:00.229000'},
 {'time_minutes': 11, 'date': '2026-03-16T16:40:27.793000'},
 {'time_minutes': 11, 'date': '2026-03-16T16:40:29.893000'},
 {'time_minutes': 12, 'date': '2026-03-16T16:40:34.450000'}]

```

Listing 27: Verificación: Hall of Fame del país 'ES'

```

[{'dungeon_id': 101,
 'dungeon_name': 'Cueva Helada',
 'top_5': [
   {'email': 'player@email.com',
    'user_name': 'Thor',
    'time_minutes': 11,
    'date': '2026-03-16T16:40:29.893000'},
   {'email': 'player@email.com',
    'user_name': 'Thor',
    'time_minutes': 12,
    'date': '2026-03-16T16:40:34.450000'}]}]]

```

Se observa que los registros se han insertado correctamente en ambas tablas y que el Hall of Fame muestra los tiempos ordenados de menor a mayor.

7.4.6. Escritura: User kills monster during Horde

Listing 28: Función user_kills_monster

```
def user_kills_monster(session, country, event_id,
                       email, user_name):
    # Paso 1: Leer n_killed actual del jugador
    # (usa el índice secundario sobre email)
    get_kills_query = """
        SELECT n_killed FROM top_horde_by_event
        WHERE country = %s AND event_id = %s
        AND email = %s;
    """
    statement = SimpleStatement(
        get_kills_query,
        consistency_level=ConsistencyLevel.ONE)
    row = session.execute(
        statement, [country, event_id, email]).one()

    # Paso 2: Calcular nuevo n_killed
    n_killed_actual = row.n_killed if row else 0
    n_killed_total = n_killed_actual + 1

    # Paso 3: DELETE + INSERT en BATCH
    batch = BatchStatement(
        consistency_level=ConsistencyLevel.ONE)

    # Borrar fila anterior si existe
    if row:
        delete_query = """
            DELETE FROM top_horde_by_event
            WHERE country = %s AND event_id = %s
            AND n_killed = %s AND email = %s;
        """
        batch.add(delete_query, (country, event_id,
                                n_killed_actual, email))

    # Insertar fila actualizada
    insert_query = """
        INSERT INTO top_horde_by_event
        (country, event_id, n_killed, email, user_name)
        VALUES (%s, %s, %s, %s, %s)
    """
    batch.add(insert_query, (country, event_id,
                             n_killed_total, email, user_name))

    # Ejecutar cambios
    try:
        session.execute(batch)
        print(f"Record de {user_name} actualizado. "
              f"Total bajas: {n_killed_total}.")
    except Exception as e:
        print(f"Error al actualizar: {e}")
```

Ejemplo de ejecución: tomamos la fila del jugador `trommleremilie` (email `kabuskarin@example.net`) en el evento 0 de Alemania, cuyo `n_killed` antes de la ejecución era 11:

```
user_kills_monster(
    session=session, country='de_DE', event_id=0,
```

```
email='kabuskarin@example.net',  
user_name='trommleremilie')
```

```
Record de trommleremilie actualizado. Total bajas: 12.
```

La función ha leído el `n_killed` actual (11), ha calculado el nuevo valor (12), ha borrado la fila antigua con `n_killed=11` y ha insertado una nueva fila con `n_killed=12`, todo dentro de un `UNLOGGED BATCH` que se ejecuta de forma atómica en la misma partición.

8. Conclusiones

El diseño en Cassandra se fundamenta en modelar las tablas según las queries de lectura, aceptando la denormalización y la duplicación de datos como compromiso necesario para conseguir lecturas eficientes sin `JOINS`. Las decisiones más relevantes del trabajo han sido:

- Usar `partition keys` compuestas que coinciden con los parámetros de entrada de cada lectura, garantizando que cada query se resuelve accediendo a una única partición.
- Emplear `clustering columns` para obtener los datos ya ordenados (tiempos ascendentes para el Hall of Fame y User Statistics, kills descendentes para la Horda) sin coste adicional de ordenación.
- Diferenciar los niveles de consistencia según el contexto de cada leaderboard: `QUORUM` para los leaderboards consultados desde el menú del campamento (donde la consistencia es importante y la latencia tolerable) y `ONE` para el leaderboard de Horda (donde la latencia es crítica durante `gameplay` y la consistencia puede ser más flexible).
- Ampliar las escrituras del enunciado original para incluir los campos denormalizados necesarios (`country`, `user_name`, `dungeon_name`), evitando lecturas adicionales en cada operación de escritura.
- Utilizar el patrón `DELETE + INSERT` dentro de un `BATCH` para actualizar `n_killed` en la tabla de Horda, aceptando una ventana mínima de inconsistencia que el propio enunciado considera aceptable e incluso beneficiosa.
- Crear un índice secundario sobre `email` en `top_horde_by_event` como solución práctica para el prototipo, siendo conscientes de que en un sistema a mayor escala convendría usar una tabla auxiliar dedicada.